

Technical Write-Up: Road Sign and Traffic Light Detection

Field	Value
Course	CSS 487 Computer Vision
Team	Ishaan, Manish Ram

1. What We Were Trying to Accomplish

The goal of this project was to build a computer vision system that can detect road signs and traffic lights from still images and dashcam style video frames. A major constraint was that the system should not rely on a pretrained machine learning model as required by the course. Instead, it should use OpenCV techniques: color segmentation, contour extraction, shape analysis, circle detection, and rule-based classification.

The intended capabilities were:

Goal	Why it mattered
Detect common US-style road signs	Demonstrates color segmentation, contour analysis, and shape classification.
Detect red, yellow, and green traffic-light bulbs	Shows that the system can recognize traffic lights, common while driving.
Run on both still images and video	Shows that the approach can process isolated examples and real driving frames.
Produce visible annotated outputs	Makes results easy to evaluate our work and to demonstrate.

2. What We Accomplished

We accomplished a working OpenCV based road sign and traffic light detection system that can process both still images and dashcam style video. The final project combines two modular pipelines: one for detecting road signs using HSV color segmentation and shape analysis, and another for detecting red, yellow, and green traffic lights using HSV masks, contour filtering, circularity checks, and Hough circle support.

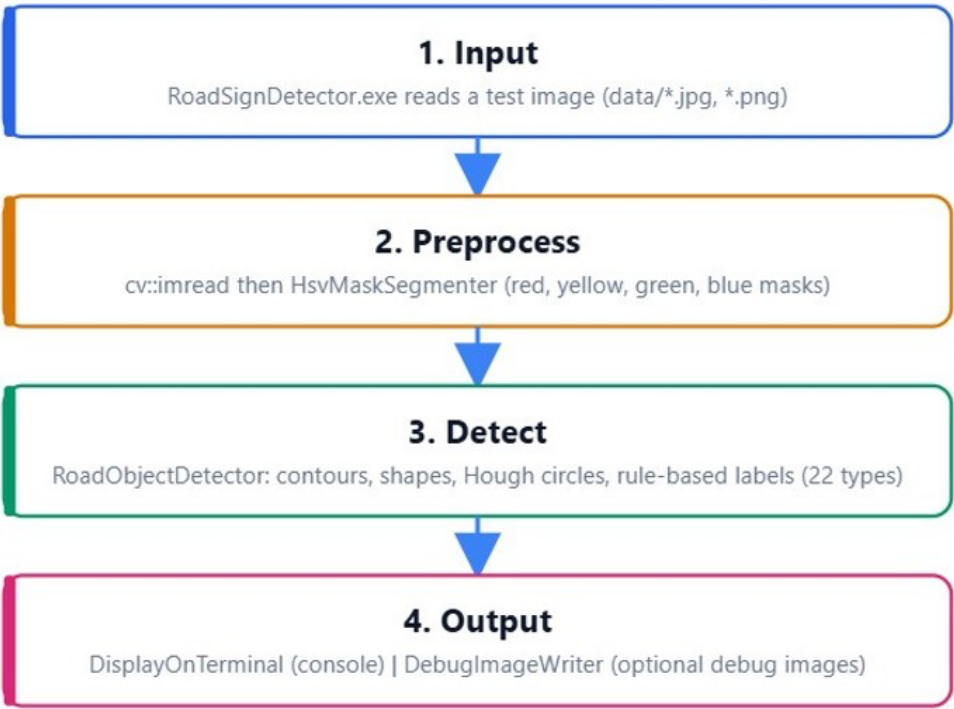
The system can identify several sign categories, including construction, guide, service, warning, stop, do-not-enter, and speed-limit signs, with speed limit examples receiving more specific digit based labels. We also added command-line modes, repeatable build/run scripts.

Completed functionality:

Area	Accomplished work
Road signs	HSV color masks plus shape rules detect construction, guide, service, warning, and regulatory signs.
Regulatory signs	Stop signs, do not enter signs, and speed limit signs receive more specific labels.
Speed-limit reading	A lightweight digit recognition path labels examples such as SPEED LIMIT 30.
Traffic lights	Red, yellow, and green bulbs are detected using HSV masks, contour filtering, and Hough-circle support.
Pipeline integration	PipelineMode controls whether the demo runs sign detection, traffic-light detection, or both.
Documentation	The repo includes this write-up, a README, presentation slides, and an architecture diagram.

3. System Design

The project uses classical image processing instead of machine learning. Each frame is converted into color masks, cleaned with morphology, filtered with contour/shape features, and then annotated with a label and bounding box.



Architecture diagram

Road-Sign Pipeline

The road sign pipeline uses computer vision techniques to detect and label common road signs from still images and video frames. Each input frame is converted from BGR to HSV color space so the system can isolate important sign colors such as red, yellow, orange, blue, green, and white. After creating these color masks, the pipeline applies morphological cleanup to reduce noise, then finds contours that may correspond to signs.

The Shape Analyzer checks each candidate using features such as contour area, bounding box size, aspect ratio, polygon shape, fill ratio, and color context. This allows the system to classify broad categories like warning, construction, guide, service, and regulatory signs. For regulatory signs, the pipeline adds extra logic to distinguish stop signs, do-not-enter signs, and speed limit signs, including a lightweight digit-reading step for speed limit examples.

Step	Method	OpenCV concepts used
Color isolation	Convert BGR to HSV and threshold sign colors	cvtColor, inRange
Mask cleanup	Remove speckle and close small holes	morphological open/close
Contour extraction	Find candidate sign blobs	findContours, boundingRect

Shape classification	Use aspect ratio, contour area, polygon shape, and fill ratio	approxPolyDP, contour geometry
Regulatory refinement	Combine red and white masks for stop, do-not-enter, and speed limit signs	nested contour and panel checks
Digit reading	Threshold the speed-limit panel and compare digit patches	threshold, matchTemplate

Traffic Light Pipeline

The traffic light pipeline detects red, yellow, and green traffic lights using HSV color segmentation, contour filtering, and circular shape analysis. Each frame is converted into HSV color space, where separate masks are created for red, yellow, and green light regions. The masks are cleaned with morphological operations, then the detector searches for candidate blobs that match the expected size, brightness, shape, and roundness of a traffic light bulb.

The system also uses Hough circle detection to confirm circular bulb like regions and compares the color mask evidence inside each candidate to decide whether it should be labeled as a red, yellow, or green traffic light. Finally, the strongest detections are selected and drawn on the frame with bounding boxes and labels, which helps reduce extra false positives from glare, reflections, or other bright objects.

Step	Method	OpenCV concepts used
HSV masking	Build red, yellow, green, and blue masks	inRange, bitwise_or
Candidate filtering	Measure area, aspect ratio, roundness, and circularity	contour features
Circle validation	Check bulb-like circular structure	HoughCircles
Label selection	Choose the dominant color within the candidate	mask pixel counts
Output filtering	Keep the strongest traffic-light detections	confidence sorting

Traffic lights are harder than signs because active bulbs are small, bright, and often surrounded by other bright objects.

4. Results

These images are annotated outputs generated by the demo and are the evidence for the current results we got.

Test group	Saved outputs	Labels demonstrated	Result summary
Construction signs	3	CONSTRUCTION SIGN	Orange construction signs are detected with rectangular/diamond-like regions.
Guide signs	4	GUIDE SIGN	Green guide signs are detected in the curated still image set.
Regulatory signs	6	STOP SIGN, DO NOT ENTER, SPEED LIMIT ##	The strongest category; stop and speed-limit examples are clearly localized.
Service signs	6	SERVICE SIGN	Blue service signs are detected across several examples.
Warning signs	4	WARNING SIGN	Yellow diamond warning signs are detected reliably in still images.
Traffic lights	10	RED_TRAFFIC_LIGHT, YELLOW_TRAFFIC_LIGHT, GREEN_TRAFFIC_LIGHT	Close/static traffic light images work best; small bright background objects can create false positives.
Dashcam video frames	5	Combined sign/light labels	Video mode runs both pipelines, but real road footage is much noisier than curated still images.

Example Outputs

Stop sign detection



Stop sign result

The detector isolates the red stop sign from a cluttered outdoor background and draws a bounding box with the STOP SIGN label.

Speed-limit detection with digit reading



Speed limit result

The speed-limit path detects the white sign panel and reads the visible number, producing the more specific SPEED LIMIT 30 label.

Warning sign detection



Warning sign result

The warning detector uses the yellow HSV mask and diamond like shape to localize the sign.

Traffic light detection



Traffic light result

The traffic-light detector finds active colored bulbs using color masks and circle like shape evidence. This example also shows a limitation: nearby bright spots can compete with the actual bulb, so the implementation keeps only the strongest few candidates.

Dashcam frame



Dashcam frame result

The dashcam frames demonstrate the combined video pipeline. They are the most challenging examples because motion blur, windshield artifacts, reflections, and distant lights can look similar to traffic light bulbs.

5. Interpretation of Results

The project succeeded as a computer vision demonstration. It detects several types of signs in curated still images, handles traffic light colors, and runs both pipelines on video. The best results occur when the target object is reasonably large, well lit, and close to the expected sign color/shape.

While modern ML is superior for production, building this pipeline manually provides an indispensable understanding of core image processing fundamentals.

Main limitations:

Limitation	Effect
Fixed HSV thresholds	Lighting changes can make signs or bulbs too dark, washed out, or shifted in hue.
Macro sign labels	Warning, service, construction, and guide signs are category labels, not exact sign meanings.
Bright-object confusion	Reflections, vehicle lights, and glare can be mistaken for traffic lights.

Simple speed-limit OCR	Works for clean two digit examples but is not robust to unusual fonts, skew, or occlusion.
No trained model	The system is explainable, but it lacks the generalization of a modern learned detector.

6. Lessons Learned

1. **Targeted detectors reduce false positives:** Running every sign detector on every image caused background colors to be mislabeled. Matching each still-image folder to its relevant detector made the demo much cleaner.
2. **HSV segmentation is powerful but fragile:** Color masks are fast and easy to debug, but they depend heavily on lighting, camera exposure, shadows, and image compression.
3. **Shape context matters:** Color alone is not enough. Contour area, aspect ratio, fill ratio, circularity, polygon approximation, and bounding-box checks helped reject many non-sign objects.
4. **Traffic lights need extra filtering:** A bright circular blob is not always a traffic light. The detector needed color evidence, circularity checks, Hough-circle support, and top-N filtering to avoid drawing too many boxes.
5. **Highly Explainable Logic:** Unlike black box neural networks, every detection in our C++ pipeline stems from a traceable geometric or color calculation. It is completely transparent.
6. **Vulnerability to Motion & Angles:** Motion blur from the car, unusual camera angles, or partially blocked signs distort the object's contours, ruining template matching capabilities.

7. Team Contributions

Member	Contributions
Ishaan	Road-sign color segmentation, sign shape analysis, regulatory-sign logic, folder demo structure, test data organization.
Manish Ram	Traffic-light segmentation, traffic-light object detection, Hough/circularity logic, detection selection, integration support.

8. How to Reproduce

From the project root on Windows:

```
scripts\build_vs2022.bat
scripts\run_demo.bat
```

To regenerate screenshots instead of opening GUI windows:

```
scripts\run_demo.bat --save-screenshots
```

Optional modes:

```
scripts\run_demo.bat --signs-only  
scripts\run_demo.bat --lights-only  
scripts\run_demo.bat --both
```

On macOS:

```
./scripts/build_macos.sh  
./scripts/run_macos.sh
```

9. References

- OpenCV 4 documentation: <https://docs.opencv.org/4.x/>
- Manual on Uniform Traffic Control Devices (MUTCD), for sign color and shape conventions
- CSS 487 course material on HSV thresholding, contour features, morphology, and Hough transforms